

Universidad Autonoma de Aguascalientes

Centro de Ciencias Basicas
Departamento de Sistemas Electronicos

Tecnologias Web

Examen Parcial 2

Mini proyecto web en equipos

Informacion general

Modalidad: Trabajo en equipos de 2 a 3 integrantes

Tiempo de desarrollo: 10 dias fecha de entrega el 17 de abril de 2026

Entrega: Proyecto desplegado, repositorio Git, reporte PDF y script SQL

Naturaleza del proyecto: Aplicacion web comercial sencilla

Proposito del examen

El objetivo de este examen es que el alumno integre, en un solo proyecto funcional, los temas vistos durante el parcial en Angular, Node.js y MySQL. Se espera que el sistema funcione como una **SPA**, consuma su **propia API**, utilice una base de datos sencilla y quede **publicado en linea**.

1. Tema general del proyecto

Cada equipo debera desarrollar una **aplicacion web comercial sencilla**. El giro del proyecto sera libre, siempre y cuando corresponda a una idea de tipo comercial y no academica.

Algunos ejemplos validos son:

- tienda de videojuegos
- catalogo de ropa
- catalogo de tenis
- tienda de accesorios
- catalogo de libros
- tienda de productos tecnologicos

Importante

El equipo puede elegir su propio giro, pero el proyecto debe mantener una estructura clara de catalogo comercial con productos, vista de detalle, carrito y seccion de contacto.

2. Objetivo

Construir una aplicacion web usando:

- **Angular** para el frontend
- **Node.js + Express** para el backend
- **MySQL** para la base de datos

La aplicacion debera funcionar como una **Single Page Application (SPA)** y consumir una **API propia** desarrollada por el mismo equipo.

3. Alcance minimo del proyecto

El proyecto sera un **catalogo web con alta de productos, carrito de compras y contacto**. Como minimo, debera incluir las siguientes vistas o secciones:

- Inicio
- Catalogo de productos
- Detalle de producto
- Formulario para agregar producto
- Carrito de compras
- Contacto

4. Funcionalidad minima esperada

4.1. Catalogo

Mostrar una lista de productos con informacion como:

- nombre

- categoria
- marca
- precio
- stock
- imagen
- descripcion corta
- disponibilidad

4.2. Detalle

Cada producto debera tener una vista de detalle usando una **ruta dinamica**. Un ejemplo de ruta valida seria:

/productos/3

4.3. Alta de productos

Debe existir un formulario funcional para registrar nuevos productos en la base de datos.

4.4. Carrito de compras

El sistema debera permitir, como minimo:

- agregar productos al carrito
- visualizar los productos agregados
- mostrar cantidad por producto
- mostrar subtotal o total general
- quitar al menos un producto del carrito

Nota sobre el carrito

No se requiere implementar pasarela de pago, cobro real ni proceso completo de compra. El carrito sera una funcionalidad basica para demostrar manejo de estado, interaccion y logica del sistema.

4.5. Contacto

Debe existir una seccion de contacto con un formulario donde el usuario pueda escribir un mensaje.

Como minimo, el formulario de contacto debera solicitar:

- nombre
- correo
- asunto
- mensaje

Ademas, al enviar el formulario debe suceder al menos una de estas dos cosas:

- guardarse el mensaje en la base de datos
- mostrarse un mensaje claro de confirmacion en pantalla

Lo ideal es que se haga **ambas cosas**: guardar el mensaje y mostrar confirmacion al usuario.

5. Base de datos

Para facilitar el desarrollo, la base de datos sera **simple y sin relaciones obligatorias entre tablas**. No se requiere el uso de llaves foraneas, joins ni un modelo relacional complejo.

5.1. Tabla principal obligatoria

Como minimo, debera existir una tabla principal, por ejemplo:

productos

con campos como los siguientes:

- id
- nombre
- categoria
- marca

- precio
- stock
- imagen
- descripcion
- disponible

5.2. Tabla de mensajes

Tambien debera existir una tabla para la seccion de contacto. Por ejemplo:

`mensajes`

con campos como:

- id
- nombre
- correo
- asunto
- mensaje

Importante

No es obligatorio relacionar estas tablas entre si. La idea es que la base de datos sea facil de construir y manipular durante el tiempo del examen.

6. Requerimientos minimos de Angular

6.1. Binding

En el proyecto deben aparecer, de manera visible y funcional, los siguientes mecanismos:

- interpolacion
- property binding
- event binding
- two-way binding

6.2. Directivas

Deben usarse al menos:

- **2 directivas estructurales**, por ejemplo: `*ngIf`, `*ngFor`
- **1 directiva de atributos**, por ejemplo: `[ngClass]` o `[ngStyle]`

6.3. Comunicacion entre componentes

Debe existir al menos un caso de comunicacion entre componentes usando:

- `@Input()`

Si ademas implementan:

- `@Output()`

se tomara en cuenta favorablemente.

6.4. Routing

El proyecto debe incluir:

- `router-outlet`
- `routerLink`
- al menos una ruta dinamica para el detalle del producto

6.5. Servicios

Debe utilizarse al menos un servicio para consumir la API propia del sistema.

Se recomienda, como minimo, un servicio para:

- productos
- carrito o estado del carrito
- mensajes de contacto

6.6. Formularios

Deben incluirse ambos enfoques:

- **1 formulario por template**
- **1 formulario reactivo**

Una manera sencilla de resolver esto es:

- formulario de contacto con template form
- formulario de alta de producto con reactive form

6.7. Pipes

Debe usarse al menos **1 pipe integrado de Angular**, por ejemplo:

- `currency`
- `uppercase`
- `titlecase`
- `date`

6.8. Signals

Debe usarse al menos **1 `signal()`** para manejar estado local. Algunos ejemplos validos son:

- contador de productos en carrito
- filtro activo
- mensaje de estado
- total del carrito

6.9. Observables

El consumo principal de datos desde Angular debera realizarse usando:

- `HttpClient`
- `Observable`
- `subscribe()`

7. Requerimientos minimos del backend

El backend debera desarrollarse con:

- Node.js
- Express
- MySQL
- CORS

7.1. Endpoints minimos

Como minimo, deben existir los siguientes endpoints:

- GET para listar productos
- GET por id para mostrar el detalle
- POST para agregar producto
- POST para guardar mensajes de contacto

Si el equipo ademas implementa PUT o DELETE, se tomara en cuenta favorablemente, pero no es obligatorio.

7.2. Middleware

Se debe implementar al menos un middleware sencillo, por ejemplo:

- validacion de campos vacios
- validacion de precio
- validacion de stock
- validacion de mensaje de contacto

8. Punto extra opcional

Como punto extra, el equipo podrá integrar **una API externa coherente con el giro elegido**. Esta API no debe sustituir la base de datos propia del sistema; únicamente podrá utilizarse para enriquecer información.

Ejemplos:

- datos complementarios de videojuegos
- portadas de libros
- información adicional de productos tecnológicos

Condición del punto extra

La funcionalidad principal del proyecto debe seguir dependiendo del backend propio y de la base de datos propia. El punto extra no sustituye los requerimientos obligatorios.

9. Deploy obligatorio

El proyecto deberá entregarse **funcionando en línea**. No basta con que funcione únicamente en local.

9.1. Se debe entregar

- link del frontend desplegado
- link del backend desplegado, si aplica
- link del repositorio GitHub
- reporte en PDF

9.2. Plataformas sugeridas

Se sugiere usar plataformas como:

- GitHub Pages para el frontend
- Render para el backend
- Railway para la base de datos

No obstante, pueden utilizar otras plataformas equivalentes si el proyecto queda correctamente publicado.

10. Requisito del repositorio

El repositorio debe mostrar **avance real del trabajo**.

Regla importante

No se aceptara un repositorio con un solo push final.

Se espera que el historial del proyecto muestre trabajo progresivo durante los 10 días de desarrollo.

10.1. Mínimo esperado

- al menos **5 commits** significativos
- realizados en distintos momentos del desarrollo
- con mensajes claros

10.2. Ejemplos de mensajes validos

- `avance routing`
- `agregado formulario reactivo`
- `conexion backend`
- `detalle por id`
- `carrito funcional`
- `deploy frontend`

11. Reporte

Cada equipo debera entregar un **reporte en PDF** con formato IEEE, pero con la siguiente consideracion:

Formato del reporte

El reporte debera tener estilo IEEE en cuanto a estructura y formalidad, **pero sin usar dos columnas**. Todo el documento debera ir en **una sola columna**.

11.1. El reporte debe incluir

- portada
- introduccion
- objetivo
- descripcion del sistema
- tecnologias utilizadas
- explicacion de la logica general
- capturas de evidencia
- conclusiones
- referencias

11.2. Contenido esperado

El reporte debe enfocarse en:

- explicar como fue construido el sistema
- describir la logica general de solucion
- mostrar resultados obtenidos
- documentar problemas encontrados y como se resolvieron

11.3. Contenido no permitido

- no debe pegarse el codigo completo
- no deben colocarse capturas largas de codigo
- 0 codigo!!

11.4. Evidencias dentro del reporte

Los **links** y las **imagenes de evidencia** deberan ir **dentro del reporte**. Por lo tanto, dentro del PDF deben aparecer:

- link del repositorio

- link del deploy
- capturas del sistema funcionando
- evidencia del carrito
- evidencia del formulario de contacto y su mensaje de confirmacion

12. Entregables finales

Cada equipo debera entregar lo siguiente (1 por equipo):

- link del repositorio GitHub
- link del proyecto desplegado
- archivo PDF del reporte
- archivo `.sql` de la base de datos

13. Criterios de evaluacion

Criterio	Porcentaje
Funcionamiento general del sistema	25 %
Uso correcto de Angular y aplicacion de los temas vistos	25 %
Backend y conexion con base de datos	20 %
Carrito de compras y contacto con mensaje	10 %
Deploy funcional	10 %
Repositorio con avance real visible	5 %
Reporte con evidencias y links	5 %
Total	100 %

14. Nota importante sobre la entrega

AulaVirtual y envío alternativo

Si el AulaVirtual presenta fallas durante el periodo de entrega, el equipo deberá enviar su trabajo a mi correo electrónico [irving.cardona@edu.uaa.mx] **dentro de las fechas correspondientes establecidas para la entrega**. El hecho de que falle AulaVirtual **no extiende automáticamente la fecha límite**; por ello, el envío por correo deberá hacerse en tiempo y forma.

Tecnologías Web — Examen Parcial 2

Ing. Irving Cardona